MPC 분산 키 생성 프로토콜 실행 과정 단계별 가이드

개요

이 문서는 TNO PET Lab의 MPC 분산 키 생성 프로토콜이 실제로 어떻게 실행되는지를 단계별로 상세히 설명합니다. 각 단계에서 일어나는 구체적인 계산과 통신 과정을 포함합니다.

1. 프로토콜 시작 및 초기 설정

1.1 참가자 풀(Pool) 설정

```
# 로컬 실행의 경우
def setup_local_pool(server_port: int, ports: List[int]) -> Pool:
pool = Pool()
pool.add_http_server(server_port) # 자신의 서버 포트 설정
for client_port in (port for port in ports if port != server_port):
pool.add_http_client(f"client{client_port}", "localhost", client_port)
return pool
```

```
# 분산 실행의 경우
def setup_distributed_pool(server_port: int, others: List[Tuple[str, int]]) ->
pool = Pool()
```

```
pool.add_http_server(server_port)

for client_ip, client_port in others:

pool.add_http_client(f"client_{client_ip}_{client_port}", client_ip, client_port

return pool
```

1.2 보안 매개변수 검증

프로토콜 시작 시 다음 조건을 확인합니다:

```
if number_of_players < 2 * corruption_threshold + 1:

raise ValueError(

f"참가자 수 {number_of_players}가 부족합니다. "

f"corruption_threshold {corruption_threshold}에 대해 "

f"최소 {2 * corruption_threshold + 1}명이 필요합니다."

)
```

이유: Shamir 비밀 분산에서 곱셈 연산 후 재구성을 위해 2t+1개의 분산이 필요하기 때문입니다.

2. 참가자 인덱스 할당 프로세스

2.1 랜덤 숫자 생성 및 교환

```
async  def  get_indices(cls, pool: Pool) -> tuple[dict[str, int], int]:

success = False

attempt = 0

while not success:
```

```
success = True
attempt += 1
# 1단계: 각자 랜덤 숫자 생성
random_number_self = randint(0, 1000000)
# 2단계: 모든 참가자에게 브로드캐스트
pool.async_broadcast(
random_number_self,
msg_id=f"distributed_keygen_random_number#{attempt}"
)
# 3단계: 다른 참가자들의 랜덤 숫자 수신
responses = await pool.recv_all(
msg_id=f"distributed_keygen_random_number#{attempt}"
)
# 4단계: 중복 검사
list_to_sort = [("self", random_number_self)]
for party, random_number_party in responses:
if random_number_party not in [rn for _, rn in list_to_sort]:
list_to_sort.append((party, random_number_party))
else:
success = False # 중복 발견 시 재시도
```

2.2 인덱스 할당 및 세션 ID 생성

```
# 5단계: 랜덤 숫자 기준으로 정렬
sorted_list = sorted(list_to_sort, key=lambda  j: j[1])
party_indices = {}
# 6단계: 정렬된 순서대로 인덱스 할당 (1부터 시작)
for index, party in  enumerate([party_name for party_name, _ in sorted_list]):
party_indices[party] = index + 1
# 7단계: 세션 ID 생성 (모든 랜덤 숫자의 합 % 1000000)
session_id = sum(i[1] for i in sorted_list) % 1000000
return party_indices, session_id
```

결과: 모든 참가자가 동일한 인덱스 매핑과 세션 ID를 가지게 됩니다.

3. Shamir 비밀 분산 스킴 초기화

3.1 소수 길이 계산

```
# 키 길이가 2048비트인 경우, p와 q는 각각 1024비트
prime_length = key_length // 2  # 1024

# Shamir 스킴을 위한 소수 길이 계산
shamir_length = 2 * (prime_length + math.ceil(math.log2(number_of_players)))
# 예: 2 * (1024 + ceil(log2(3))) = 2 * (1024 + 2) = 2052
```

3.2 두 가지 Shamir 스킴 생성

```
# t-out-of-n 스킴 (일반적인 비밀 분산용)
shamir_scheme_t = Shamir(
sympy.nextprime(2**shamir_length), # 2052비트보다 큰 첫 번째 소수
number_of_players, # 참가자 수
corruption_threshold # t
)

# 2t-out-of-n 스킴 (곱셈 후 재구성용)
shamir_scheme_2t = Shamir(
sympy.nextprime(2**shamir_length),
number_of_players,
corruption_threshold * 2  # 2t
)
```

4. 분산 소수 생성 과정

4.1 후보 소수 생성 (각 참가자 독립적으로)

```
async  def  generate_prime_candidate(prime_length: int, prime_list: list[int]) -
while True:
# 1단계: 랜덤 홀수 생성
candidate = secrets.randbits(prime_length)
candidate |= (1 << (prime_length - 1)) | 1  # 최상위 비트와 최하위 비트를 1로 설정
```

```
# 2단계: 작은 소수들로 나누어 확인
if any(candidate % p == 0 for p in prime_list):
continue
# 3단계: Miller-Rabin 소수성 테스트
if sympy.isprime(candidate):
return candidate
```

4.2 분산 소수성 검증

각 후보 소수에 대해 분산 방식으로 소수성을 검증합니다:

```
async def distributed_primality_test(
candidate: int, generators: list[int], index: int,
pool: Pool, party_indices: dict[str, int], session_id: int
) -> bool:
# 1단계: 각 생성자에 대해 Jacobi 기호 계산
jacobi_results = []
for g in generators:
jacobi_symbol = sympy.jacobi_symbol(g, candidate)
jacobi_results.append(jacobi_symbol == 1)
# 2단계: 결과를 다른 참가자들과 교환
message_id = f"primality_test_session#{session_id}_candidate#{candidate}"
pool.async_broadcast(jacobi_results, msg_id=message_id)
# 3단계: 다른 참가자들의 결과 수신
```

```
responses = await pool.recv_all(msg_id=message_id)

# 4단계: 모든 참가자의 결과가 일치하는지 확인

all_results = [jacobi_results] + [response[1] for response in responses]

# 모든 참가자가 동일한 결과를 가져야 함

for i, results in  enumerate(all_results[1:]):

if results != jacobi_results:

return False

# 모든 생성자에 대해 Jacobi 기호가 1이어야 함

return  all(jacobi_results)
```

5. 분산 모듈러스 계산

5.1 소수 분산

각 참가자가 자신의 소수 p와 q를 Shamir 비밀 분산으로 분산합니다:

```
# 참가자 i가 소수 p_i를 분산

p_shares = shamir_scheme_t.share_secret(p_i)



# 각 참가자 j에게 해당하는 분산 전송

for j, share in p_shares.shares.items():

if j != index:

pool.asend(f"party_{j}", {

"type": "p_share",
```

```
"share": share,
"from": index
}, msg_id=f"p_sharing_session#{session_id}")
```

5.2 분산 곱셈 수행

모든 참가자의 p와 q 분산을 받은 후, 분산 곱셈을 수행합니다:

```
# 모든 p 분산의 합 계산
p_total_share = sum(received_p_shares.values()) % shamir_scheme_t.prime

# 모든 q 분산의 합 계산
q_total_share = sum(received_q_shares.values()) % shamir_scheme_t.prime

# 분산 곱셈: N = P × Q
# 각 참가자가 자신의 분산끼리 곱함
n_share = (p_total_share * q_total_share) % shamir_scheme_t.prime
```

5.3 N 재구성

2t+1개의 N 분산을 사용하여 최종 모듈러스 N을 재구성합니다:

```
# N 분산들을 교환
pool.async_broadcast(n_share, msg_id=f"n_reconstruction_session#{session_id}")
n_shares_dict = await pool.recv_all(msg_id=f"n_reconstruction_session#{session_i
```

```
# 라그랑주 보간을 사용하여 N 재구성

n_shares = ShamirShares(shamir_scheme_2t, n_shares_dict)

N = n_shares.reconstruct_secret()
```

6. 분산 이중 소수 검증

생성된 N이 정확히 두 소수의 곱인지 검증합니다:

```
async def distributed_biprimality_test(

n: int, generators: list[int], index: int,

pool: Pool, party_indices: dict[str, int], session_id: int

) -> bool:

test_results = []

for g in generators:

# 각 참가자가 g^((n-1)/2) mod n 계산

exponent = (n - 1) // 2

result = pow(g, exponent, n)

# 결과가 ±1인지 확인 (이차잉여 판정)

test_results.append(result in [1, n-1])

# 결과를 다른 참가자들과 교환하여 일치성 확인

message_id = f"biprimality_test_session#{session_id}_n#{n}"

pool.async_broadcast(test_results, msg_id=message_id)
```

```
responses = await pool.recv_all(msg_id=message_id)

# 모든 참가자의 결과가 일치하고, 모든 테스트를 통과해야 함

for _, other_results in responses:

if other_results != test_results:

return False

return  all(test_results)
```

7. 비밀키 매개변수 계산

7.1 λ(N) 분산 계산

카마이클 함수 λ(N) = lcm(p-1, q-1)을 분산 방식으로 계산:

```
# 각 참가자의 (p_i - 1) 분산

p_minus_1_shares = {}

for i, p_share in p_shares.items():

p_minus_1_shares[i] = p_share - 1  if i == 1  else p_share


# 각 참가자의 (q_i - 1) 분산

q_minus_1_shares = {}

for i, q_share in q_shares.items():

q_minus_1_shares[i] = q_share - 1  if i == 1  else q_share


# gcd((p-1), (q-1)) 분산 계산 (유클리드 호제법 사용)
```

```
gcd_shares = distributed_gcd(p_minus_1_shares, q_minus_1_shares)


# λ(N) = (p-1)(q-1) / gcd(p-1, q-1) 계산

lambda_shares = distributed_division(

distributed_multiply(p_minus_1_shares, q_minus_1_shares),

gcd_shares

)
```

7.2 β 값 계산

```
# β = λ(N) mod N 계산

beta_shares = {}

for i, lambda_share in lambda_shares.items():

beta_shares[i] = lambda_share % N
```

7.3 θ 값 계산

```
# g = N + 1 (Paillier 암호화의 기본 생성자)

g = N + 1


# L(x) = (x - 1) / N 함수 정의

def  L(x: int) -> int:

return (x - 1) // N
```

```
# θ = L(g^λ mod N²)^(-1) mod N 계산

g_lambda_mod_n_square = pow(g, lambda_reconstructed, N * N)

l_result = L(g_lambda_mod_n_square)

theta = pow(l_result, -1, N) # 모듈러 역원
```

8. 분산 복호화 프로세스

8.1 부분 복호화 수행

각 참가자가 자신의 비밀키 분산을 사용하여 부분 복호화를 수행:

```
def  partial_decrypt(self, ciphertext: PaillierCiphertext) -> int:

# 라그랑주 보간 계수 계산

other_players = [j for j in  range(1, self.share.degree + 2) if j != self.player

# 분자: Π(j) for j in other_players

numerator = 1

for j in other_players:

numerator *= j

# 분모: Π(j - i) for j in other_players

denominator = 1

for j in other_players:

denominator *= (j - self.player_id)

# 지수 계산: n! * numerator * share / denominator
```

```
n_factorial = math.factorial(self.share.degree + 1)

exponent = (n_factorial * numerator * self.share.shares[self.player_id]) // deno

# 부분 복호화: c^exponent mod N²

ciphertext_value = ciphertext.get_value()

if exponent < 0:

ciphertext_value = pow(ciphertext_value, -1, self.n_square)

exponent = -exponent

partial_decryption = pow(ciphertext_value, exponent, self.n_square)

return partial_decryption
```

8.2 부분 복호화 결과 교환

```
# 자신의 부분 복호화 결과를 다른 참가자들에게 전송

encryption_hash = bin(ciphertext.peek_value()).zfill(32)[2:34]

message_id = f"distributed_decryption_session#{self.session_id}_hash#{encryption


self.pool.async_broadcast({

"content": "partial_decryption",

"value": partial_decryption

}, msg_id=message_id)


# 다른 참가자들의 부분 복호화 결과 수신
```

```
other_partial_decryptions = await  self.pool.recv_all(msg_id=message_id)
```

8.3 최종 복호화

```
def  decrypt(self, partial_dict: dict[int, int]) -> int:

# 모든 부분 복호화 결과를 곱함

combined_decryption = 1

for i in  range(self.share.degree + 1):

combined_decryption *= partial_dict[i + 1]

combined_decryption %= self.n_square

# L 함수 적용: L(combined_decryption) = (combined_decryption - 1) / N

if (combined_decryption - 1) % self.n != 0:

raise ValueError("복호화 실패: 일관성 검사 실패")

l_result = (combined_decryption - 1) // self.n

# 최종 메시지: L(combined_decryption) * θ^(-1) mod N

message = (l_result * self.theta_inv) % self.n

return message
```

9. 에러 처리 및 재시도 메커니즘

9.1 소수 생성 실패 시

```
max_attempts = 100
```

```
for attempt in range(max_attempts):
try:
p_candidate = await generate_prime_candidate(prime_length, prime_list)
if await distributed_primality_test(p_candidate, generators, index, pool, party
p = p_candidate
break
except Exception as e:
logger.warning(f"소수 생성 시도 {attempt + 1} 실패: {e}")
if attempt == max_attempts - 1:
raise RuntimeError("소수 생성 최대 시도 횟수 초과")
```

9.2 통신 오류 처리

```
async def safe_broadcast_with_retry(pool: Pool, message: Any, msg_id: str, max
for retry in range(max_retries):
try:
pool.async_broadcast(message, msg_id=msg_id)
return
except Exception as e:
logger.warning(f"브로드캐스트 시도 {retry + 1} 실패: {e}")
if retry == max_retries - 1:
raise
```

```
    await asyncio.sleep(2 ** retry) # 지수 백오프
```

10. 성능 최적화 기법

10.1 배치 처리

```
# 여러 후보 소수를 동시에 생성하여 통신 오버헤드 감소
async def generate_prime_batch(batch_size: int, prime_length: int) -> list[int
candidates = []
while len(candidates) < batch_size:
candidate = await generate_prime_candidate(prime_length, prime_list)
if await distributed_primality_test(candidate, generators, index, pool, party_i
candidates.append(candidate)
return candidates
```

10.2 병렬 처리

```
# p와 q를 병렬로 생성
p_task = asyncio.create_task(generate_prime_batch(batch_size, prime_length))
q_task = asyncio.create_task(generate_prime_batch(batch_size, prime_length))

p_candidates, q_candidates = await asyncio.gather(p_task, q_task)
```

이 가이드는 MPC 분산 키 생성 프로토콜의 실제 실행 과정을 단계별로 상세히 설명하여, 프로토콜의 동작 원리를 완전히 이해할 수 있도록 도와줍니다.